FairGuard AI

Project documentation

FairGuard AI is a Streamlit workbench for evaluating a binary classification model from several responsible-AI perspectives. It uses the UCI Adult Income dataset as a reproducible demonstration dataset and compares an unmitigated baseline with a Fairlearn demographic-parity model. The application also supports evaluation of an uploaded or pre-built third model.

The workbench is an assessment and education tool. Its results are not a deployment approval for hiring, lending, credit, or other high-impact decisions.

1. What the project provides

FairGuard presents seven dashboard areas:

1. **Data Explorer** — dataset size, feature count, class balance, group distributions, and observed positive rates.
2. **Model Comparison** — accuracy, AUC, F1, demographic-parity difference, equalized-odds difference, disparate impact, and error-rate disparity.
3. **Fairness Audit** — per-group metrics, group sizes, small-group flags, confusion matrices, and intersectional analysis.
4. **Explainability** — global SHAP importance, local waterfall explanations, SHAP group differences, and proxy-feature warnings.
5. **Drift & Privacy** — Evidently data-drift analysis, k-anonymity screening, and membership-inference proxy results.
6. **Governance** — model-card generation, downloads, scorecards, hyperparameters, and a deployment checklist.
7. **Responsible AI Review** — equalized-odds approval gate, reviewer name, reason, and persisted decision status.

2. Repository layout

```
app.py                          Streamlit dashboard
src/data_loader.py              Dataset loading, cleaning, and profiling
src/models.py                   Preprocessing, model training, prediction,
metrics
src/fairness_engine.py          Fairness metrics and group visualizations
src/xai_engine.py               SHAP explanations and proxy checks
src/drift_engine.py             Evidently drift report generation
src/privacy_engine.py           k-anonymity and membership-inference proxy
src/model_card.py               Markdown and HTML model-card generation
src/governance.py               Approval and decision persistence
```

```
data/adult_income.csv        Local Adult Income dataset
input/                       Pre-built Model C artifacts
artifacts/                   Generated reports, models, and MLflow data
full_test.py                 Full integration test
smoke_test.py                Short integration test
data.py ... privacy.py       Standalone module smoke runners
```

The `artifacts/mlruns/` directory contains MLflow file-store data produced during training. It is operational tracking output, not application source code.

3. Installation

Use Python 3.10–3.12. Create an isolated environment and install the pinned project dependencies:

```
python -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

On Windows, activate with:

```
.venv\\Scripts\\activate
```

The main dependencies are Streamlit, pandas, NumPy, scikit-learn, Fairlearn, SHAP, Evidently, MLflow, matplotlib, joblib, and Markdown.

4. Running the dashboard

From the repository root:

```
streamlit run app.py
```

The application loads the local dataset, performs a stratified 80/20 split, trains Model A and Model B, and evaluates them. The first run can take several minutes because model training, SHAP computation, drift analysis, and report generation are performed lazily or on demand.

The sidebar is the source manager. A pre-built `.joblib` model can be selected, or a custom `.joblib` / `.pkl` file can be uploaded. A custom file must contain a `(model, preprocessor)` tuple whose preprocessor accepts the project feature columns.

5. Data flow

`src.data_loader.load()` performs these steps:

1. Read `data/adult_income.csv` when it exists.
2. Otherwise fetch the Adult dataset through OpenML and save a local CSV.
3. Drop missing rows and sample up to 20,000 records.

4. Normalize `sex` and `race` strings.
5. Create `age_bin` with `<30`, `30-50`, and `>50` categories.
6. Return `X`, `y`, and a separate sensitive-attribute frame.

The model input `X` deliberately excludes `sex`, `race`, `age_bin`, `income`, and the original `class` label. The sensitive columns remain in the separate `sens` frame for auditing only. This separation prevents those attributes from being direct model inputs while allowing measured model behavior to be compared across cohorts.

Set `FAIRGUARD_SAMPLE=1` for a faster development run. This limits the loader to 1,000 rows. Unset it for the real-data run.

6. Models

Model A: baseline

Model A is a `HistGradientBoostingClassifier`. Categorical columns are one-hot encoded and numeric columns are standardized through a `ColumnTransformer`. The model is trained without sensitive columns.

Model B: fairness-aware model

Model B uses Fairlearn `ExponentiatedGradient` with logistic regression and a `DemographicParity` constraint. The training split's `sex` values are supplied to Fairlearn as sensitive features for the reduction algorithm. They are not included in the feature matrix.

Model C: external model

Model C is optional. It is loaded from the sidebar and evaluated with the same held-out test rows. Every dashboard area that supports model comparison includes Model C when it is active.

7. Performance metrics

The dashboard reports:

- **Accuracy** — fraction of correct predictions.
- **AUC** — ROC area using positive-class scores where available.
- **F1** — harmonic mean of precision and recall.

Metrics are calculated on the held-out test split. They should be interpreted with the dataset limitations and class balance in mind.

8. Fairness metrics

For each selected sensitive attribute, `fairness_engine.audit()` computes per-group:

- selection rate;

- true-positive rate (TPR);
- false-positive rate (FPR);
- false-negative rate (FNR);
- false-discovery rate (FDR);
- error rate;
- group count;
- a small-group flag when fewer than 100 test rows are present.

The summary metrics are:

- **Demographic-parity difference** — maximum selection rate minus minimum selection rate.
- **Equal-opportunity gap** — maximum TPR minus minimum TPR.
- **Equalized-odds difference** — equal-opportunity gap plus the absolute FPR gap.
- **Disparate impact** — minimum selection rate divided by maximum selection rate.
- **Error-rate disparity** — maximum group error rate minus minimum group error rate.

Small groups should be treated cautiously because rates computed from few rows are unstable. The group count is displayed beside fairness tables.

9. Explainability

SHAP calculations run on the preprocessed feature matrix, then aggregate one-hot contributions back to the original feature columns. This makes the returned explanation columns match the model input columns.

The Explainability tab provides:

- global beeswarm plot;
- global bar importance plot;
- local waterfall plot for a selected test row;
- absolute SHAP differences by `sex` cohort.

The proxy warning checks the ten highest-ranked SHAP features. It triggers when `relationship`, `marital-status`, or their transformed names appear among those features. This is a review signal, not proof that a feature is unfair or impermissible.

10. Drift monitoring

The drift engine compares the training reference frame with a shifted test frame. The demonstration shift adds five years to `age` and three hours to `hours-per-week`, within reasonable caps.

The generated result includes:

- drifted feature share;
- total feature count;
- per-feature test name, score, and drift flag;
- `artifacts/drift.html`, an Evidently report.

The project supports the older Evidently 0.4 API and the newer public API layout found in later installations. The report extraction code handles both formats.

11. Privacy screening

k-anonymity

The k-anonymity check groups rows by available quasi-identifiers: age, sex, race, education, and marital status. It reports minimum k, the percentage of rows with k below five, and `unique_pct`, the percentage of rows with k equal to one.

Privacy risk thresholds are:

- **Low:** unique percentage below 1%;
- **Medium:** 1% through 5%;
- **High:** above 5%.

This is a screening measure. It is not a formal privacy guarantee.

Membership-inference proxy

The MIA proxy compares train and test confidence and AUC. It labels risk from the train/test AUC gap: below 0.03 is Low, below 0.08 is Medium, and 0.08 or higher is High.

12. Governance and approval

The generated model card contains these sections: Model Details, Intended Use, Out-of-scope Use, Evaluation Data, Performance Metrics, Fairness & Privacy Findings, Explainability, Privacy, Drift, Limitations, Ethical notes, and Approval status.

The Responsible AI Review tab applies two approval conditions:

1. equalized-odds gap must be at most 0.1;
2. reviewer name must be non-empty.

Approved or rejected decisions are appended to `artifacts/decisions.json` with model, status, reviewer, reason, and UTC time. When no decision exists, the current status is `Pending`.

13. Testing

Fast integration test:

```
export FAIRGUARD_SAMPLE=1
python smoke_test.py
```

Full integration test:

```
python full_test.py
```

Standalone module checks:

```
python data.py
python train.py
python fairness.py
python explain.py
python drift.py
python privacy.py
```

Final real-data check:

```
unset FAIRGUARD_SAMPLE
python train.py
streamlit run app.py
```

The Streamlit test harness can render the dashboard without a browser:

```
from streamlit.testing.v1 import AppTest

app = AppTest.from_file("app.py").run(timeout=600)
assert not app.exception
assert len(app.tabs) == 7
```

14. Troubleshooting

MLflow tracking warning

If an existing file-store run is stale or incompatible, model training continues and artifacts are still saved. MLflow logging is supplementary to the assessment pipeline. A clean tracking directory or a migrated database backend can be used for production tracking.

SHAP takes a long time

SHAP explanations are permutation-based for the generic estimator interface. Use `FAIRGUARD_SAMPLE=1` during development, reduce the explanation sample size for local work, or wait for the first cached dashboard run.

OpenML cannot be reached

Keep `data/adult_income.csv` in place for offline operation. If the CSV is missing, the loader needs network access to fetch the Adult dataset.

Uploaded model fails

Confirm the file contains exactly `(model, preprocessor)`. Confirm that the preprocessor accepts the twelve non-sensitive project feature columns and that the estimator supports `predict()`.

Model warnings after loading joblib files

Joblib artifacts should be generated with a compatible scikit-learn version. Re-run `generate_input_models.py` after changing the environment or feature schema.

15. Responsible-use limits

The Adult dataset reflects historical 1994 US Census data. It may encode historical measurement and sampling limitations. Fairness metrics depend on group definitions, thresholds, sample sizes, and the selected evaluation split. Results should be reviewed by domain, legal, privacy, and governance specialists before any real deployment decision.

16. Maintainer checklist

- Keep sensitive attributes separate from `X`.
- Regenerate bundled input models after feature-schema changes.
- Run the fast suite after each code change.
- Run the full suite with the sample flag unset before release.
- Inspect drift and privacy artifacts before presenting a demo.
- Review the model card and persisted approval decision.
- Keep generated MLflow runs and reports out of source commits unless explicitly needed.